

1. Introducción

La evolución de la ingeniería de software ha impulsado el uso de enfoques que reduzcan la separación existente entre la especificación de requisitos, el diseño del sistema y su implementación. Cuando estas etapas se desarrollan como actividades independientes, los cambios realizados en los requisitos o en los modelos pueden no reflejarse correctamente en el código fuente, generando inconsistencias, retrabajo y defectos. En este contexto, la Ingeniería Dirigida por Modelos o Model-Driven Engineering (MDE) considera los modelos como artefactos centrales del proceso de desarrollo, mientras que el Model-Driven Development (MDD) aplica este principio para obtener, total o parcialmente, artefactos de implementación mediante transformaciones automatizadas. La arquitectura MDA organiza este proceso mediante los niveles CIM, PIM y PSM, permitiendo avanzar desde la representación del dominio hasta modelos vinculados con una plataforma tecnológica específica[1],[2].

La aplicación de este enfoque requiere que los requisitos sean claros, verificables y trazables. La norma ISO/IEC/IEEE 29148:2018 establece principios para documentar y gestionar requisitos a lo largo del ciclo de vida del software, de manera que puedan relacionarse con los modelos de diseño, los componentes implementados y sus respectivas pruebas. De forma complementaria, UML proporciona una notación normalizada para representar la estructura y el comportamiento del sistema mediante clases, asociaciones, casos de uso, secuencias y otros elementos[3]. En un proceso MDD, estos modelos dejan de ser únicamente representaciones gráficas y se convierten en entradas procesables para transformaciones modelo-a-modelo y modelo-a-texto, estas últimas orientadas a producir código fuente, configuraciones o documentación[4],[3].

Esta relación resulta pertinente para el proyecto MundiPets, concebido como una plataforma web orientada a facilitar procesos de adopción y cruce responsable de mascotas. Su especificación de requisitos de software contempla la gestión de perfiles de mascotas, historiales médicos, documentos veterinarios, publicaciones, solicitudes de adopción o cruce, comunicación entre usuarios, privacidad de la información, validación profesional y componentes de inteligencia artificial. Asimismo, el proyecto identifica actores como propietarios, adoptantes, interesados en cruce, refugios y veterinarias, junto con requisitos de seguridad, disponibilidad, usabilidad, integridad y arquitectura modular. Estos elementos ofrecen una base apropiada para construir un modelo de clases compuesto por entidades como "Usuario", "Mascota", "HistorialMedico", "DocumentoVeterinario", "Publicacion", "Solicitud", "Mensajes", "ValidacionVeterinaria", conservando la correspondencia con los requisitos previamente especificados.

Dentro de este escenario, BOUML puede emplearse como herramienta CASE para transformar parte del diseño conceptual de MundiPets en artefactos de implementación. La herramienta permite representar clases, atributos, operaciones, visibilidades, asociaciones, generalizaciones y cardinalidades, elementos necesarios para definir la estructura estática del subsistema seleccionado. Un estudio desarrollado por Durai et al. utilizó BOUML para exportar diagramas UML mediante un esquema XMI, del cual se extrajeron clases, miembros, métodos y

llamadas necesarias para generar código automáticamente[5]. Este antecedente demuestra que los modelos creados con BOUML pueden actuar como una representación formal e intercambiable, capaz de alimentar procesos de generación de código y mantener una correspondencia explícita entre los elementos del modelo y los archivos resultantes.

Aplicado a MundiPets, el diagrama de clases desarrollado en BOUML podría emplearse para generar la estructura inicial de las entidades del dominio, incluyendo sus atributos, métodos y relaciones. Por ejemplo, una asociación entre Mascota e HistorialMedico debería reflejarse en el código mediante referencias o colecciones tipadas; de igual manera, las operaciones definidas en las clases podrían convertirse en firmas de métodos dentro del lenguaje objetivo. Este procedimiento permitiría disminuir la duplicación de trabajo, mantener convenciones estructurales uniformes y evidenciar la trazabilidad modelo-código exigida en la actividad. Además, el proceso de roundtrip controlado permitiría modificar un atributo, operación o cardinalidad en el modelo, regenerar el código y verificar que el cambio aparezca en el artefacto correspondiente, conforme a los criterios establecidos en la guía de evaluación.

La literatura reciente respalda el potencial de estas prácticas, pero también señala que la generación automática no elimina la necesidad de intervención técnica. Las herramientas basadas en modelos elevan el nivel de abstracción y pueden reducir parte del código escrito manualmente, aunque no todo desarrollo visual constituye necesariamente un proceso MDD completo[6]. Otros trabajos han demostrado que las transformaciones UML y modelo-a-texto pueden producir implementaciones coherentes y reducir errores repetitivos, siempre que existan reglas de transformación, estereotipos y mecanismos de validación claramente definidos[7],[8]. Una revisión sistemática reciente también identifica al paradigma modelo-a-código como una de las estrategias predominantes de generación automática, pero advierte sobre limitaciones relacionadas con la validación del código, el soporte multilenguaje y la necesidad de comprobar empíricamente los resultados obtenidos.

Por consiguiente, el uso de BOUML en MundiPets no pretende generar automáticamente la totalidad de la aplicación. Aspectos como las reglas de adopción y cruza, el control de acceso por roles, la protección de datos, la validación documental, la persistencia, las interfaces web y los componentes de inteligencia artificial requerirán configuración adicional o implementación manual. Su valor principal reside en convertir el modelo UML en una fuente estructurada para la generación inicial del código, facilitar la correspondencia entre requisitos, clases y artefactos de software, y permitir que los cambios del diseño puedan propagarse de forma controlada. En consecuencia, el objetivo del presente trabajo es demostrar la relación entre BOUML y el PFC MundiPets mediante el modelado de un subsistema representativo, la configuración del mecanismo de generación, la obtención y verificación del código fuente y la ejecución de un cambio controlado que evidencie la trazabilidad entre el modelo de origen y la implementación resultante.

2. Marco teórico

2.1. Model-Driven Engineering (MDE)

La *Model-Driven Engineering* (MDE), o Ingeniería Dirigida por Modelos, es un paradigma de ingeniería de software en el que los modelos constituyen artefactos principales del proceso de desarrollo y no únicamente representaciones gráficas de apoyo. Bajo este enfoque, los modelos describen la estructura, el comportamiento y las restricciones de un sistema con un nivel de abstracción superior al código fuente, mientras que las transformaciones automatizadas permiten obtener otros modelos, código, configuraciones o documentación. La literatura reciente destaca que MDE comparte con las plataformas *low-code* el uso de representaciones visuales y automatización; sin embargo, MDE se diferencia por el empleo explícito de lenguajes de modelado, metamodelos, reglas de transformación y mecanismos de trazabilidad entre artefactos [9, 10].

En el PFC MundiPets, este paradigma se aplica al utilizar la Especificación de Requisitos de Software como punto de partida para construir un modelo UML que represente el dominio del sistema. La plataforma contempla procesos relacionados con perfiles de mascotas, historial médico, documentos veterinarios, publicaciones, solicitudes de adopción o cruce, mensajería, validación profesional y control de privacidad. Estos elementos pueden expresarse mediante clases, atributos, operaciones y asociaciones dentro de BOUML, de modo que el modelo conserve una correspondencia verificable con los requisitos funcionales y las restricciones de diseño del PFC.

Por tanto, el valor de BOUML dentro de MundiPets no se limita a dibujar el diagrama de clases. El modelo construido en la herramienta actúa como una representación estructurada del subsistema seleccionado y como fuente para la generación de artefactos de implementación. Esta relación permite que elementos como `Mascota`, `Usuario`, `HistorialMedico`, `DocumentoVeterinario`, `Publicacion` y `Solicitud` mantengan atributos tipados, operaciones, visibilidades y cardinalidades coherentes antes de trasladarse al código. En consecuencia, MDE aporta a MundiPets una forma sistemática de conservar la trazabilidad entre requisitos, modelo UML y estructura del software generado.

2.3. Model-Driven Development (MDD)

El *Model-Driven Development* (MDD), o Desarrollo Dirigido por Modelos, corresponde a la aplicación práctica de los principios de MDE durante la construcción del software. En MDD, el modelo de origen guía la implementación y es procesado por generadores o transformaciones modelo-a-texto para producir código fuente de manera total o parcial. La automatización busca disminuir tareas repetitivas, evitar inconsistencias estructurales y mantener reglas uniformes de implementación. No obstante, el código generado debe ser verificado, compilado y complementado cuando el modelo no contiene suficiente información para representar toda la lógica del negocio [11].

Un antecedente directamente relacionado con la herramienta seleccionada es

el trabajo de Durai *et al.*, quienes emplearon BOUML para exportar modelos UML mediante XMI y procesar elementos como clases, miembros, métodos y llamadas durante la generación automática de código [12]. Este estudio evidencia que un modelo creado en BOUML puede ser utilizado como entrada de un proceso de generación y no solamente como documentación visual. La calidad del resultado depende de la precisión del modelo, de las reglas de transformación utilizadas y de la correcta interpretación de las relaciones UML.

En MundiPets, MDD se materializa al seleccionar un subsistema representativo, modelarlo en BOUML y generar la estructura inicial de sus clases. Por ejemplo, el módulo de gestión de mascotas e información veterinaria puede incluir las clases `Usuario`, `Mascota`, `HistorialMedico`, `Vacuna`, `DocumentoVeterinario`, `ValidacionVeterinaria` y `Publicacion`. A partir del modelo, BOUML puede producir declaraciones de clases, atributos, operaciones y relaciones que sirvan como base del proyecto. Sin embargo, aspectos como la autenticación, la autorización por roles, la persistencia, las interfaces web, la moderación y los componentes de inteligencia artificial requieren configuración adicional o desarrollo manual. Por esta razón, el equipo debe distinguir claramente el código generado del código agregado posteriormente y comprobar que la estructura obtenida conserva fidelidad con el modelo UML de origen.

2.9. Forward, reverse y round-trip engineering

El *forward engineering* es el proceso mediante el cual un modelo se transforma en código fuente u otros artefactos de implementación. El *reverse engineering* realiza el recorrido contrario, debido a que analiza código existente para recuperar su estructura y representarla en un modelo. Por su parte, el *round-trip engineering* combina ambos sentidos con el propósito de mantener una correspondencia controlada entre el modelo y el código cuando alguno de los dos cambia. La investigación reciente muestra que este proceso puede incluir la modificación de estructuras en el código y su posterior reimportación a una herramienta gráfica; no obstante, su eficacia depende de la capacidad del entorno para analizar el código, preservar información y resolver diferencias entre artefactos [13].

En el caso de MundiPets, el *forward engineering* se evidencia cuando el diagrama de clases elaborado en BOUML genera las clases del subsistema seleccionado. El *reverse engineering* puede emplearse para recuperar la estructura de código existente y contrastarla con el diseño UML, siempre que el lenguaje objetivo y la configuración usada por BOUML sean compatibles. El cierre del ciclo debe demostrarse mediante un *round-trip* controlado: se modifica en el modelo un elemento significativo, como agregar el atributo `estadoValidacion` a `DocumentoVeterinario`, incorporar una operación o cambiar una cardinalidad; posteriormente se regenera el código y se verifica que el cambio aparezca en el archivo correspondiente.

Este procedimiento no debe interpretarse como una sincronización automática perfecta. Los estudios industriales recientes identifican la gestión de la consistencia entre modelos y código como uno de los principales retos del desarrollo basado en modelos, especialmente cuando existen ciclos cortos de cambio, múltiples

herramientas y código manual [14]. En consecuencia, el equipo de MundiPets debe conservar copias de los artefactos antes y después de la regeneración, registrar la política de sobrescritura de BOUML y comprobar que el código agregado manualmente no sea eliminado. Las capturas del modelo, del código y del resultado de la compilación constituyen la evidencia necesaria para demostrar la trazabilidad del ciclo.

2.11. Estado del arte

El estado actual de la Ingeniería Dirigida por Modelos muestra una evolución desde los generadores tradicionales de esqueletos de código hacia entornos que integran lenguajes específicos de dominio, plataformas *low-code*, transformaciones automáticas, verificación de consistencia y flujos DevOps. Las revisiones recientes señalan que las plataformas visuales y MDE presentan objetivos comunes, como elevar el nivel de abstracción y reducir la programación repetitiva, aunque difieren en el grado de formalización de los modelos y en la transparencia de sus mecanismos de generación [9, 10]. También se observa que MDE se aplica en dominios complejos mediante técnicas como transformaciones modelo-a-modelo, generación de código e interpretación de modelos [15].

A pesar de estos avances, continúan abiertos problemas relacionados con la interoperabilidad entre herramientas, la conservación de la trazabilidad, la validación del código generado, la escalabilidad de las transformaciones y la sincronización entre modelos y código. La evidencia industrial indica que los procesos de desarrollo continuo incrementan la necesidad de mecanismos automáticos de consistencia, debido a que los cambios frecuentes pueden separar rápidamente el diseño de la implementación [14]. Por ello, una herramienta MDD debe evaluarse no solo por su capacidad para producir archivos, sino también por la calidad del modelo de entrada, la reproducibilidad del generador, la posibilidad de compilar el resultado y la protección del trabajo manual.

Dentro de este panorama, BOUML representa una herramienta CASE ligera y aplicable a la generación estructural desde UML. Su uso reciente como fuente de modelos XMI para generación automática demuestra que puede participar en un flujo MDD verificable [12]. Para MundiPets, BOUML resulta adecuado para modelar un subsistema del dominio y generar la base de sus clases, manteniendo una correspondencia visible entre requisitos, entidades, atributos, operaciones y relaciones. Sin embargo, su aplicación debe complementarse con un IDE, un compilador, control de versiones y verificaciones manuales para cubrir la lógica de negocio, la seguridad, la persistencia y los componentes de inteligencia artificial. Así, el aporte principal de BOUML al PFC consiste en reducir el trabajo estructural repetitivo y proporcionar una evidencia reproducible de la transformación UML–código, sin asumir que la herramienta genera por sí sola la aplicación completa.

3. Justificación de la selección de BOUML

3.1. Herramienta seleccionada y versión

La herramienta seleccionada para desarrollar la demostración de Ingeniería Dirigida por Modelos aplicada al PFC MundiPets es **BOUML 7.11, parche 4**. Esta versión fue escogida por ofrecer un entorno UML orientado al modelado estructural, la generación de código, la ingeniería inversa y el *round-trip engineering*. BOUML es una herramienta UML 2 de uso gratuito y multiplataforma, con soporte para diferentes lenguajes de programación, entre ellos Java [16, 17].

Para la demostración del proyecto se configurará C++ como lenguaje objetivo. BOUML se empleará para construir el diagrama de clases del subsistema seleccionado y generar la estructura inicial de sus clases. El código resultante servirá como base para la implementación posterior del backend de MundiPets. El alcance de la herramienta se concentrará en los elementos que pueden derivarse directamente del modelo UML, como clases, atributos, operaciones, visibilidades y relaciones. La interfaz web, la persistencia, las reglas de negocio y los componentes de inteligencia artificial deberán configurarse o implementarse posteriormente.

La versión seleccionada permite trabajar con un entorno estable y reproducible durante la exposición. No obstante, el equipo deberá registrar en el repositorio la versión exacta utilizada, el sistema operativo, el lenguaje objetivo, el directorio de generación y cualquier configuración necesaria. De esta manera, otra persona podrá abrir el modelo, ejecutar nuevamente el generador y comprobar los resultados obtenidos.

3.2. Justificación frente a los requisitos del PFC

La selección de BOUML se realizó considerando el dominio de MundiPets, el lenguaje objetivo, la arquitectura modular, la licencia, la curva de aprendizaje y la necesidad de demostrar trazabilidad entre el modelo UML y el código fuente, tal como exige la guía de la actividad [18]. MundiPets es una plataforma web orientada a facilitar procesos de adopción y cruce responsable de mascotas. Su ERS contempla la gestión de perfiles de mascotas, historiales médicos, carnés de vacunación, documentos veterinarios, publicaciones, solicitudes, validaciones profesionales, privacidad y funcionalidades de apoyo mediante inteligencia artificial [19].

Correspondencia con el dominio modelado. El subsistema seleccionado puede representarse mediante clases como **Usuario**, **Mascota**, **HistorialMedico**, **CarnetVacunacion**, **DocumentoVeterinario**, **ValidacionVeterinaria**, **Publicacion** y **Solicitud**. Estas clases se relacionan directamente con requisitos del PFC como registrar mascotas, gestionar información médica, validar documentos sanitarios, administrar publicaciones y procesar solicitudes de adopción o cruce. BOUML permite definir atributos tipados, operaciones, visibilidades, asociaciones, generalizaciones y cardinalidades, por lo que resulta adecuado para construir un diagrama de clases no trivial y coherente con el dominio.

Generación de código. La herramienta permite convertir parte del modelo UML en código fuente, reduciendo la escritura repetitiva de estructuras básicas. La generación no sustituye la implementación completa del sistema, ya que el código obtenido debe ser compilado, verificado y complementado con la lógica que no pueda derivarse del modelo [11]. Un estudio reciente empleó modelos exportados desde BOUML mediante XMI para extraer clases, miembros y métodos y generar código automáticamente, lo que respalda su utilización como fuente de artefactos de implementación [12].

Arquitectura modular. La ERS de MundiPets establece que el sistema debe organizarse mediante una arquitectura modular para facilitar su mantenimiento y la incorporación de funcionalidades de adopción, cruza, validación veterinaria e inteligencia artificial [19]. En BOUML, las clases pueden organizarse mediante paquetes y vistas que representen los módulos del dominio. Esta organización facilita que el código generado mantenga una separación inicial de responsabilidades.

Trazabilidad y cierre del ciclo. BOUML permite demostrar el flujo modelo-código requerido en la actividad: construir el modelo, generar el código, modificar un atributo, una operación o una relación, regenerar y verificar que el cambio se refleje en el archivo correspondiente [17]. Sin embargo, la consistencia entre modelos y código no debe asumirse como automática. Estudios industriales recientes señalan que mantener ambos artefactos sincronizados continúa siendo un reto cuando existe código manual o cambios frecuentes [14]. Por ello, el equipo deberá conservar evidencias antes y después de cada regeneración y registrar los cambios mediante commits en GitHub.

Licencia y curva de aprendizaje. BOUML puede utilizarse sin adquirir una licencia comercial, lo que facilita su instalación en los equipos de los integrantes. Su interfaz está centrada en UML y generación de código, por lo que el equipo puede concentrarse en el diagrama de clases, la configuración del generador y la verificación del resultado. La curva de aprendizaje es moderada, debido a que requiere comprender la organización del proyecto, las vistas, los artefactos y los *plug-outs*; aun así, resulta proporcionada para el alcance de la demostración. Esta decisión mantiene el principio de MDD según el cual el modelo constituye un artefacto operativo y no solamente un dibujo [9].

En consecuencia, BOUML satisface las necesidades principales de la actividad: permite modelar las clases relevantes de MundiPets, generar código C++, realizar ingeniería inversa y demostrar un cambio controlado entre modelo y código. Su selección es adecuada siempre que se diferencie claramente el código producido por la herramienta del código agregado manualmente.

3.3. Comparación con alternativa descartada

La alternativa analizada fue **Modelio 5.4.1**. Modelio es un entorno de modelado de código abierto que admite UML y otros lenguajes o marcos de modelado, como BPMN, ArchiMate y SysML. También dispone de módulos para generación de código e ingeniería inversa [20]. Por tanto, Modelio no fue descartado por falta de capacidad, sino porque sus funciones adicionales exceden

el alcance inmediato de la demostración planificada para MundiPets. Modelio habría sido una alternativa razonable si el trabajo incluyera BPMN, SysML, arquitectura empresarial o una estrategia de modelado más extensa. Sin embargo, la actividad actual debe demostrar de forma reproducible la transformación de un diagrama de clases de MundiPets en código C++, su verificación y un cambio controlado modelo-código. Para este alcance, BOUML ofrece un flujo más directo y una menor carga de configuración. La decisión no implica que BOUML sea superior en todos los contextos, sino que resulta más proporcionado para el subsistema y las evidencias exigidas.

Cuadro 1: Comparación entre BOUML y Modelio para el PFC MundiPets.

Criterio	BOUML 7.11, parche 4	Modelio 5.4.1	Decisión para MundiPets
Licencia y costo	Uso gratuito; no requiere adquirir una licencia para la demostración.	Núcleo de código abierto y disponible sin costo.	Ambos cumplen; el costo no constituye el criterio diferenciador.
Generación Java	Incluye herramientas para generación, ingeniería inversa y <i>round-trip</i> en Java.	La generación y la ingeniería inversa se incorporan mediante módulos.	Se prefiere BOUML por ofrecer un flujo directo para la demostración seleccionada.
Alcance funcional	Entorno concentrado en UML, generación de código y utilidades relacionadas.	Entorno más amplio, con UML, BPMN, ArchiMate, SysML y servicios adicionales.	Las capacidades extra de Modelio no son necesarias para el subsistema que se generará.
Configuración	Permite configurar el lenguaje, los artefactos y el directorio de salida en el mismo proyecto.	Requiere seleccionar y documentar los módulos empleados para la generación.	BOUML reduce la cantidad de componentes que deben prepararse y explicarse.
Adecuación al PFC	Suficiente para modelar clases del dominio de MundiPets y generar su estructura en C++ o Java.	Adecuado para proyectos que requieran una cobertura más amplia de procesos y arquitectura.	BOUML presenta una relación más directa entre el diagrama de clases elegido y el código que será verificado.
Evolución	La versión seleccionada prioriza estabilidad, aunque presenta menor proyección de actualización.	Mantiene un ecosistema más amplio y orientado a diferentes tipos de modelado.	Modelio ofrece ventajas para una estrategia de evolución más extensa; BOUML se conserva por el alcance académico inmediato.

4. Descripción del subsistema del PFC modelado

4.1. Contexto de MundiPets

MundiPets se constituye como una plataforma digital integral orientada a facilitar y organizar los procesos de adopción y cruce responsable de mascotas. La propuesta responde a la necesidad de disponer de un canal centralizado, transparente y seguro que conecte a propietarios de mascotas, adoptantes, interesados

en cruza, refugios de animales y veterinarias.

A través de la plataforma se busca centralizar y digitalizar la gestión de perfiles biológicos de los animales, el almacenamiento de historiales clínicos, la publicación de anuncios y la verificación de certificados veterinarios. De esta manera, MundiPets organiza la interacción entre los usuarios que desean dar o recibir una mascota en custodia, quienes buscan un emparejamiento biológico controlado y los profesionales veterinarios encargados de validar la información sanitaria registrada.

4.2. Módulo seleccionado

El componente seleccionado para trabajar con la herramienta BOUML corresponde al subsistema de **Gestión de Mascotas e Historial Médico**. Este módulo es fundamental dentro de MundiPets, debido a que administra la información básica, clínica y de control de los animales antes de habilitar otros procesos de la plataforma, como la adopción, la cruza responsable o la validación de información médica.

El módulo se encuentra conformado por un conjunto de clases relacionadas que representan el dominio del sistema. La clase base **Usuario** se especializa en las clases **Propietario** y **Veterinario**. La entidad central es **Mascota**, asociada con **HistorialMedico** mediante una relación de composición. Además, se incluyen el catálogo **Vacuna**, la clase asociativa **MascotaVacuna**, encargada de representar el carné sanitario, y la clase **EvaluacionVeterinaria**, destinada a registrar las revisiones profesionales de salud.

Por medio de estas clases, el subsistema permite registrar los datos generales de los animales, almacenar antecedentes clínicos, controlar las dosis de vacunas aplicadas y gestionar las evaluaciones veterinarias correspondientes.

4.3. Actores

Los actores que interactúan directamente con el subsistema seleccionado se definen a partir de los roles representados en la jerarquía del modelo UML:

- **Propietario:** usuario encargado de registrar y administrar las mascotas dentro de la plataforma. Sus acciones principales incluyen ingresar la información general del animal, actualizar sus datos básicos, registrar antecedentes médicos iniciales y gestionar sus publicaciones.
- **Veterinario:** profesional encargado de revisar y validar la información de salud. Dentro del módulo puede consultar historiales clínicos, revisar documentos oficiales de respaldo y registrar las evaluaciones veterinarias correspondientes.

4.4. Requisitos funcionales

El alcance funcional del subsistema comprende las operaciones de registro, actualización, consulta y validación profesional de la información biológica y

Cuadro 2: Requisitos funcionales del subsistema de Gestión de Mascotas e Historial Médico.

ID	Requisito	Descripción
RF-01	Registrar mascota	El sistema deberá permitir al propietario registrar una mascota ingresando su información general, incluyendo nombre, especie, raza, sexo, edad, estado reproductivo y fotografías para su identificación dentro de la plataforma.
RF-02	Gestionar historial médico	El sistema deberá permitir registrar y actualizar el historial médico de una mascota, incluyendo vacunas, desparasitaciones, enfermedades, alergias, cirugías, tratamientos y controles veterinarios.
RF-03	Consultar perfil completo	El sistema deberá permitir visualizar el perfil completo de una mascota, mostrando su información general, historial médico autorizado, fotografías, comportamiento, vacunas, edad y raza para procesos de adopción o cruce.
RF-09	Validar información médica	El sistema deberá permitir que una veterinaria autorizada valide la información médica registrada de una mascota e indique que los datos han sido verificados por un profesional.
RF-14	Gestionar carné de vacunación	El sistema deberá permitir registrar, actualizar y consultar el carné de vacunación de cada mascota, mostrando el historial de vacunas aplicadas, sus fechas, las próximas dosis y el estado de vigencia.
RF-16	Gestionar antecedentes genéticos	El sistema deberá permitir registrar y consultar los antecedentes genéticos y el grado de parentesco de una mascota, incluyendo enfermedades hereditarias y linaje, con el fin de apoyar los procesos de cruce responsable y la evaluación de compatibilidad.

clínica de las mascotas. Los requisitos funcionales vinculados con el módulo se resumen en la Tabla 2.

La caracterización completa de cada requisito, incluyendo sus atributos de gestión y su especificación atómica, se encuentra recopilada en el Anexo A del documento principal.

4.5. Alcance del módulo modelado

El trabajo realizado con BOUML se enfoca exclusivamente en la definición de la estructura estática del subsistema de Gestión de Mascotas e Historial Médico. Por esta razón, el alcance se divide en elementos incluidos y excluidos.

4.5.1. Elementos incluidos

- Definición visual de las clases involucradas, con sus nombres, atributos tipados y firmas de métodos públicos o privados.
- Representación de las relaciones UML del dominio, incluyendo generalización, composición y asociaciones con sus respectivas multiplicidades.
- Modelado de relaciones complejas mediante clases asociativas, como **MascotaVacuna**, utilizada para representar el control del carné de vacunación.

4.5.2. Elementos excluidos

- Desarrollo de la lógica interna o programación detallada del cuerpo de los métodos y funciones de negocio.
- Configuración física de esquemas, tablas o conexiones con la base de datos.
- Diseño visual o maquetación de las pantallas de la interfaz gráfica de usuario.

Referencias

- [1] International Organization for Standardization and International Electrotechnical Commission and Institute of Electrical and Electronics Engineers, *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, ISO/IEC/IEEE Std. ISO/IEC/IEEE 29 148:2018, Nov. 2018. [Online]. Available: <https://www.iso.org/standard/72089.html>
- [2] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*, Object Management Group Std. formal/2008-01-16, Jan. 2008. [Online]. Available: <https://www.omg.org/spec/MOFM2T/1.0>
- [3] —, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, Object Management Group Std. formal/2017-12-05, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>
- [4] —, “Model driven architecture guide, revision 2.0,” Object Management Group, Tech. Rep. ormsc/2014-06-01, Jun. 2014. [Online]. Available: <https://www.omg.org/cgi-bin/doc?ormsc/2014-06-01>
- [5] A. D. Durai, M. Ganesh, R. M. Mathew, and D. K. Anguraj, “A novel approach with an extensive case study and experiment for automatic code generation from the XMI schema of UML models,” *The Journal of*

- Supercomputing*, vol. 78, no. 6, pp. 7677–7699, 2022. [Online]. Available: <https://doi.org/10.1007/s11227-021-04164-x>
- [6] D. D. Ruscio, D. S. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022. [Online]. Available: <https://doi.org/10.1007/s10270-021-00970-2>
 - [7] L. Jiménez-Navajas, R. Pérez-Castillo, and M. Piattini, “Code generation for classical-quantum software systems modeled in UML,” *Software and Systems Modeling*, vol. 24, no. 3, pp. 795–821, 2025. [Online]. Available: <https://doi.org/10.1007/s10270-024-01259-w>
 - [8] L. Huning and E. Pulvermüller, “Automatic code generation of safety mechanisms in model-driven development,” *Electronics*, vol. 10, no. 24, p. 3150, 2021. [Online]. Available: <https://doi.org/10.3390/electronics10243150>
 - [9] D. D. Ruscio, D. S. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022. [Online]. Available: <https://doi.org/10.1007/s10270-021-00970-2>
 - [10] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, “Modelling in low-code development: A multi-vocal systematic review,” *Software and Systems Modeling*, vol. 21, no. 5, pp. 1959–1981, 2022. [Online]. Available: <https://doi.org/10.1007/s10270-021-00964-0>
 - [11] L. Huning and E. Pulvermüller, “Automatic code generation of safety mechanisms in model-driven development,” *Electronics*, vol. 10, no. 24, p. 3150, 2021. [Online]. Available: <https://doi.org/10.3390/electronics10243150>
 - [12] A. D. Durai, M. Ganesh, R. M. Mathew, and D. K. Anguraj, “A novel approach with an extensive case study and experiment for automatic code generation from the XMI schema of UML models,” *The Journal of Supercomputing*, vol. 78, no. 6, pp. 7677–7699, 2022. [Online]. Available: <https://doi.org/10.1007/s11227-021-04164-x>
 - [13] U. Kannengiesser, J. Frysak, C. Stary, F. Krenn *et al.*, “Developing an engineering tool for cyber-physical production systems,” *e & i Elektrotechnik und Informationstechnik*, vol. 138, pp. 330–340, 2021. [Online]. Available: <https://doi.org/10.1007/s00502-021-00911-3>
 - [14] R. Jongeling, F. Ciccozzi, J. Carlson, A. Cicchetti *et al.*, “Consistency management in industrial continuous model-based development settings: A reality check,” *Software and Systems Modeling*, vol. 21, pp. 1511–1530, 2022. [Online]. Available: <https://doi.org/10.1007/s10270-022-01000-5>

- [15] D. Lehner, J. Zhang, J. Pfeiffer, S. Sint, A.-K. Splettstößer, M. Wimmer, and A. Wortmann, “Model-driven engineering for digital twins: A systematic mapping study,” *Software and Systems Modeling*, vol. 24, pp. 1339–1377, 2025. [Online]. Available: <https://doi.org/10.1007/s10270-025-01264-7>
- [16] B. Pagès, “BOUML: A free UML tool box,” Sitio web oficial de BOUML, 2025, consultado en julio de 2026. [Online]. Available: <https://www.bouml.fr/>
- [17] —, “BOUML features,” Documentación oficial de BOUML, 2026, consultado en julio de 2026. [Online]. Available: <https://www.bouml.fr/features.html>
- [18] G. G. Ulloa, “Guía y rúbrica de evaluación: Herramientas que generan código fuente a partir de modelos uml,” Universidad Técnica Estatal de Quevedo, Quevedo, Ecuador, Guía académica, 2026.
- [19] Equipo MundiPets, “Proyecto fin de curso: Entrega 1b—documento ers/srs parcial,” Universidad Técnica Estatal de Quevedo, Quevedo, Ecuador, Especificación de Requisitos de Software, 2026, versión 2.0.
- [20] Modelio Open Source, “Modelio: The open source modeling environment,” Sitio web oficial de Modelio, 2026, versión 5.4.1. Consultado en julio de 2026. [Online]. Available: <https://www.modelio.org/>